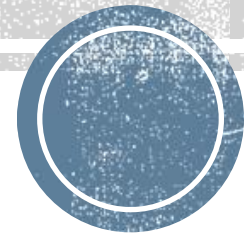


Функции и процедуры. Язык PL/pgSQL

Программирование на стороне сервера БД



Функции

В Postgres Pro представлены функции четырёх видов:

- функции на языке запросов (функции, написанные на SQL);
- функции на процедурных языках (функции, написанные, например, на PL/pgSQL);
- внутренние функции;
- функции на языке C.

Функции любых видов могут принимать в качестве аргументов (параметров) базовые типы, составные типы или их сочетания. Кроме того, любые функции могут возвращать значения базового или составного типа. Также можно определить функции, возвращающие наборы базовых или составных значений.

Проще всего определить функции на языке SQL, поэтому сначала мы рассмотрим их. Многие концепции, касающиеся функций на SQL, затем распространятся и на другие виды функций.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Общие сведения



Объект базы данных

определение хранится в системном каталоге

Основные составные части определения

имя

параметры

тип возвращаемого значения

тело

Доступны несколько языков, в том числе SQL

код в виде строковой константы

обычно интерпретируется при вызове

Вызывается в контексте выражения



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Функции являются такими же объектами базы данных, как, например, таблицы и индексы.

Определение функции сохраняется в системном каталоге; поэтому функции в базе данных называют хранимыми.

В PostgreSQL доступно большое количество стандартных функций, с некоторыми из них можно познакомиться в справочном материале «Основные типы данных и функции».

И, конечно, можно писать собственные функции на разных языках программирования. Материал этой темы относится к функциям на любом языке, но примеры будут использовать язык SQL.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Определение функции состоит из имени, необязательных параметров, типа возвращаемого значения и тела.

Тело записывается в виде строковой константы, которая содержит код на выбранном языке.

За счет этого определение функции выглядит одинаково независимо от выбранного языка.



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Функция всегда вызывается в контексте какого-либо выражения. Например, в списке выражений команды SELECT, в условии WHERE, в ограничении целостности CHECK и т. п.

<https://postgrespro.ru/docs/postgresql/16/sql-createfunction>

<https://postgrespro.ru/docs/postgresql/16/sql-syntax-calling-funcs>



Функции, определяемые на SQL (функции SQL или SQL-функции)



Функции SQL с базовыми типами

Простейшая возможная функция SQL не имеет аргументов и просто возвращает базовый тип, например integer:

```
CREATE FUNCTION one() RETURNS integer AS $$  
    SELECT 1 AS result;  
$$ LANGUAGE SQL;
```

```
-- Альтернативная запись строковой константы:  
CREATE FUNCTION one() RETURNS integer AS '  
    SELECT 1 AS result;  
' LANGUAGE SQL;
```

```
SELECT one();
```

```
one  
-----  
1
```

Мы определили псевдоним столбца в теле функции для её результата (дали ему имя result), но этот псевдоним не виден снаружи функции. Вследствие этого, столбец результата получил имя one, а не result.



Функции SQL с базовыми типами

Практически так же легко определяются функции SQL, которые принимают в аргументах базовые типы:

```
CREATE FUNCTION add_em(x integer, y integer) RETURNS integer AS $$  
    SELECT x + y;  
$$ LANGUAGE SQL;
```

```
SELECT add_em(1, 2) AS answer;
```

```
answer
```

```
-----
```

```
3
```



Функции SQL с базовыми типами

Мы также можем отказаться от имён аргументов и обращаться к ним по номерам:

```
CREATE FUNCTION add_em(integer, integer) RETURNS integer AS $$  
    SELECT $1 + $2;  
$$ LANGUAGE SQL;
```

```
SELECT add_em(1, 2) AS answer;
```

answer

3



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Функции без параметров

Вот простой пример функции без параметров, содержащей один оператор:

```
=> CREATE FUNCTION hello_world() -- имя и пустой список параметров
    RETURNS text                  -- тип возвращаемого значения
    AS $$ SELECT 'Hello, world!'; $$ -- тело
    LANGUAGE sql;                 -- указание языка
```

```
CREATE FUNCTION
```

Тело удобно записывать в строке, заключенной в кавычки-доллары, как в приведенном примере. Иначе придется заботиться об экранировании кавычек, которые наверняка встретятся в теле функции. Сравните:

```
=> SELECT ' SELECT ''Hello, world!''; ';
```

```
           ?column?
-----
SELECT 'Hello, world!';
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

```
=> SELECT $$ SELECT 'Hello, world!'; $$;
```

```
      ?column?
-----
SELECT 'Hello, world!';
(1 row)
```

При необходимости кавычки-доллары могут быть вложенными. Для этого в каждой паре кавычек надо использовать разный текст между долларами:

```
=> SELECT $func$ SELECT $$Hello, world!$$; $func$;
```

```
      ?column?
-----
SELECT $$Hello, world!$$;
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Функция вызывается в контексте выражения, например:

```
=> SELECT hello_world(); -- пустые скобки обязательны
```

```
hello_world
-----
Hello, world!
(1 row)
```

Давайте взглянем на то, как тело функции хранится в системном каталоге.

```
=> \pset xheader_width 60
```

```
Expanded header width is 60.
```

```
=> SELECT proname, prosrc, prosqlbody FROM pg_proc
WHERE proname = 'hello_world' \gx
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

```
-[ RECORD 1 ]-----  
proname      | hello_world  
prosrc       | SELECT 'Hello, world!';  
prosqlbody   |
```

Мы видим сохраненную в исходном виде тело-строку.

А теперь реализуем современную возможность пересоздать нашу функцию в другом виде — в стиле стандарта SQL. В нашем случае телом функции может быть один оператор вида RETURN <выражение>:

```
=> CREATE OR REPLACE FUNCTION hello_world() RETURNS text  
LANGUAGE sql  
RETURN 'Hello, world!';
```

```
CREATE FUNCTION
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

В случае, если тело функции состоит из нескольких операторов SQL, в качестве результата возвращается значение из первой строки, которую вернул последний оператор. Если код такой функции написан в стиле стандарта SQL, потребуется использовать конструкцию BEGIN ATOMIC ... END, которая охватывает выполняемый блок операторов:

```
=> CREATE OR REPLACE FUNCTION hello_world() RETURNS text
LANGUAGE sql
BEGIN ATOMIC
    SELECT 'First Line';
    SELECT 'Second Line';
END;
```

CREATE FUNCTION

Пробуем вызов:

```
=> SELECT hello_world();

hello_world
-----
Second Line
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Обратите внимание на особенности синтаксиса стиля стандарта SQL — в отличие от традиционного «строчного»:

- нет конструкции AS, содержащей код функции в виде строки;
- может использоваться новое ключевое слово RETURN для возврата значения;
- указание LANGUAGE sql не является обязательным;
- при создании функции ее код разбирается, а результат разбора сохраняется в pg_proc.prosqlbody (в традиционной нотации текст функции сохраняется в pg_proc.prosrc).

Это лучше соответствует стандарту и в большей мере совместимо с другими реализациями SQL. Теперь при вызове функции ее команды заново не интерпретируются, а используется заранее разобранный вариант.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Функции с входными параметрами

Пример функции с одним параметром:

```
=> CREATE FUNCTION hello(name text) -- формальный параметр
    RETURNS text
    LANGUAGE sql
    RETURN 'Hello, ' || name || '!';
```

CREATE FUNCTION

При вызове функции мы указываем фактический параметр, соответствующий формальному:

```
=> SELECT hello('Alice');
```

```
      hello
-----
Hello, Alice!
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

При указании типа параметра можно указать и модификатор (например, `varchar(10)`), но он игнорируется.

Можно определить параметр функции без имени; тогда внутри тела функции на параметры придется ссылаться по номеру. Удалим функцию и создадим новую:

```
=> DROP FUNCTION hello(text); -- достаточно указать тип параметра
```

```
DROP FUNCTION
```

```
=> CREATE FUNCTION hello(text)
RETURNS text
LANGUAGE sql
RETURN 'Hello, ' || $1 || '!'; -- номер вместо имени
```

```
CREATE FUNCTION
```

```
=> SELECT hello('Alice');
```

```
      hello
-----
Hello, Alice!
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Но так лучше не делать, это неудобно.

Удалим функцию и создадим заново, добавив еще два параметра — приветствие и обращение.

```
=> DROP FUNCTION hello(text);
```

DROP FUNCTION

Здесь мы используем необязательное ключевое слово IN, обозначающее входной параметр. Предложение DEFAULT позволяет определить значение по умолчанию для параметра:

```
=> CREATE FUNCTION hello(IN name text, IN greet text DEFAULT 'Dear', IN title text DEFAULT 'Mr')
RETURNS text
LANGUAGE sql
RETURN 'Hello, ' || greet || ' ' || title || ' ' || name || '!';
```

CREATE FUNCTION



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

```
=> SELECT hello('Alice', 'Charming', 'Mrs'); -- указаны второй и третий параметры
```

```
      hello
-----
Hello, Charming Mrs Alice!
(1 row)
```

Обратите внимание, что параметры со значениями по умолчанию должны идти в конце всего списка. При вызове функции значения фактических параметров, определенных как default, можно опускать, тогда остальные default-параметры, идущие в списке после, также получают значения по умолчанию

```
=> SELECT hello('Bob', 'Excellent'); -- указан только первый default-параметр
```

```
      hello
-----
Hello, Excellent Mr Bob!
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

```
=> SELECT hello('Bob'); -- опущены оба параметра, имеющие значение по умолчанию
```

```
      hello
-----
Hello, Dear Mr Bob!
(1 row)
```

До сих пор мы вызывали функцию, указывая фактические параметры позиционным способом — в том порядке, в котором они определены при создании функции. Во многих стандартных функциях имена параметров не заданы, так что этот способ оказывается единственным.

Но если формальным параметрам даны имена, можно использовать их при указании фактических параметров. В этом случае параметры могут указываться в произвольном порядке:

```
=> SELECT hello(title => 'Mrs', name => 'Alice');
```

```
      hello
-----
Hello, Dear Mrs Alice!
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Такой способ удобен, когда порядок параметров неочевиден, особенно если их много; также можно явно указать значение одного из параметров по умолчанию.

Можно совмещать оба способа: часть параметров (начиная с первого) указать позиционно, а оставшиеся — по имени:

```
=> SELECT hello('Alice', title => 'Mrs');
```

```
         hello
-----
Hello, Dear Mrs Alice!
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Если функция должна возвращать неопределенное значение, когда хотя бы один из входных параметров не определен, ее можно объявить как строгую (STRICT). Тело функции при этом вообще не будет выполняться.

```
=> DROP FUNCTION hello(text, text, text);
```

```
DROP FUNCTION
```

```
=> CREATE FUNCTION hello(IN name text, IN title text DEFAULT 'Mr')  
RETURNS text  
LANGUAGE sql STRICT  
RETURN 'Hello, ' || title || ' ' || name || '!';
```

```
CREATE FUNCTION
```

```
=> SELECT hello('Alice', NULL);
```

```
hello  
-----
```

```
(1 row)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Каждой функции сопоставлена категория изменчивости, которая определяет свойства возвращаемого значения при одинаковых значениях входных параметров.

Категория `volatile` говорит о том, что возвращаемое значение может произвольно меняться. Такие функции будут выполняться каждый раз при каждом вызове. Если при создании функции категория не указана, назначается именно эта категория.

Категория `stable` используется для функций, возвращаемое значение которых не меняется в пределах одного SQL-оператора. В частности, такие функции не могут менять состояние БД. Такая функция *может* быть выполнена один раз во время выполнения запроса, а затем будет использоваться вычисленное значение.



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_o8_sql_func.html

Категория `immutable` еще более строгая: возвращаемое значение не меняется никогда. Такую функцию можно выполнить на этапе планирования запроса, а не во время выполнения.

Можно — не означает, что всегда происходит именно так, но планировщик вправе выполнить такие оптимизации. В некоторых (простых) случаях планировщик делает собственные выводы об изменчивости функции, невзирая на указанную явно категорию.

<https://postgrespro.ru/docs/postgresql/16/xfunc-volatility>



Функции SQL со сложными типами

В функциях *с аргументами составных типов* мы должны указывать не только, какой аргумент, но и какой атрибут (поле) этого аргумента нам нужен.

Например, предположим, что emp — таблица, содержащая данные работников, и это же имя составного типа, представляющего каждую строку таблицы.

Следующая функция double_salary вычисляет, каким было бы чьё-либо жалование в случае увеличения вдвое:

```
CREATE TABLE emp (  
    name      text,  
    salary    numeric,  
    age       integer,  
    cubicle   point  
);  
INSERT INTO emp VALUES ('Bill', 4200, 45, '(2,1)');  
  
CREATE FUNCTION double_salary(emp) RETURNS numeric AS $$  
    SELECT $1.salary * 2 AS salary;  
$$ LANGUAGE SQL;
```



Функции SQL со сложными типами

```
SELECT name, double_salary(emp.*) AS dream  
FROM emp  
WHERE emp.cubicle ~= point '(2,1)';
```

name		dream
------	--	-------

-----+-----		
-------------	--	--

Bill		8400
------	--	------

Запись \$1.salary позволяет выбрать одно поле из значения строки аргумента. В вызывающей команде SELECT указание * выбирает всю текущую строку таблицы как составное значение. На строку таблицы можно сослаться и просто по имени таблицы, например так:

```
SELECT name, double_salary(emp) AS dream  
FROM emp  
WHERE emp.cubicle ~= point '(2,1)';
```

Однако это использование считается устаревшим, так как провоцирует путаницу.



Функции SQL со сложными типами

Иногда бывает удобно образовать составное значение аргумента на лету. Это позволяет сделать конструкция ROW. Например, так можно изменить данные, передаваемые в функцию:

```
SELECT name, double_salary(ROW(name, salary*1.1, age, cubicle)) AS dream  
FROM emp;
```

Также возможно создать функцию, возвращающую составной тип. Например, эта функция возвращает одну строку emp:

```
CREATE FUNCTION new_emp() RETURNS emp AS $$  
  SELECT text 'None' AS name,  
         1000.0 AS salary,  
         25 AS age,  
         point '(2,2)' AS cubicle;  
$$ LANGUAGE SQL;
```

В этом примере для каждого атрибута задано постоянное значение, но вместо этих констант можно подставить любые вычисления.



Функции SQL со сложными типами

Есть два важных требования относительно определения функции:

- Порядок в списке выборки внутреннего запроса должен в точности совпадать с порядком следования колонок в таблице, связанной с составным типом. (Имена колонок для системы значения не имеют.)
- Следует привести выражения в соответствие с определением составного типа, либо можно получить такие ошибки:

ERROR: function declared to return emp returns varchar instead of text at column 1

(ОШИБКА: функция, объявленная как возвращающая emp, возвращает varchar вместо text в колонке 1)

Ту же функцию можно определить другим способом:

```
CREATE FUNCTION new_emp() RETURNS emp AS $$  
    SELECT ROW('None', 1000.0, 25, '(2,2)')::emp;  
$$ LANGUAGE SQL;
```

Здесь записан SELECT, который возвращает одну колонку нужного составного типа. В данной ситуации этот вариант на самом деле не лучше, но в некоторых случаях он может быть удобной альтернативой — например, если нам нужно вычислить результат, вызывая другую функцию, которая возвращает нужное составное значение.



Функции SQL со сложными типами

Можно вызвать эту функцию напрямую двумя способами:

```
SELECT new_emp();
```

```
new_emp
-----
(None,1000.0,25,"(2,2)")
```

```
SELECT * FROM new_emp();
```

```
name | salary | age | cubicle
-----+-----+-----+-----
None | 1000.0 | 25 | (2,2)
```

Когда используется функция, возвращающая составной тип, может возникнуть желание получить из её результата только одно поле (атрибут). Это можно сделать, применяя такую запись:

```
SELECT (new_emp()).name;
```

```
name
-----
None
```

Дополнительные скобки необходимы во избежание неоднозначности при разборе запроса. Если попытаться выполнить запрос без них, возникнет ошибка:

```
SELECT new_emp().name;
ERROR: syntax error at or near "."
LINE 1: SELECT new_emp().name;
```



Функции SQL со сложными типами

В качестве ещё одного варианта можно использовать функциональную запись для извлечения атрибута: запись атрибут(таблица) и таблица.атрибут взаимозаменяемы.

```
SELECT name(new_emp());
```

```
name
```

```
-----
```

```
None
```

```
SELECT name(emp) AS youngster FROM emp WHERE age(emp) < 30;
```

```
--То же самое:
```

```
-- SELECT emp.name AS youngster FROM emp WHERE emp.age < 30;
```

```
youngster
```

```
-----
```

```
Sam
```

```
Andy
```

Подсказка: Равнозначность функциональной записи и записи с атрибутом позволяет использовать функции с составными типами для имитации "вычисляемых полей". Например, используя предыдущее определение `double_salary(emp)`, мы можем записать

```
SELECT emp.name, emp.double_salary FROM emp;
```

Приложению, использующему такой запрос, не обязательно определённо знать, что `double_salary` — это не настоящая колонка таблицы. (Вычисляемые поля также можно имитировать в представлениях.)

Вследствие такого поведения, функции, которая принимает один аргумент составного типа, не стоит давать имя, совпадающее с именем одного из полей этого составного типа.



Функции SQL со сложными типами

Ещё один вариант использования функции, возвращающей составной тип, заключается в передаче её результата другой функции, которая принимает этот тип строки на вход:

```
CREATE FUNCTION getname(emp) RETURNS text AS $$  
    SELECT $1.name;  
$$ LANGUAGE SQL;
```

```
SELECT getname(new_emp());
```

```
getname  
-----  
None  
(1 row)
```

И ещё один способ использовать функцию, возвращающую составной тип — вызвать её как табличную функцию (см. далее).



Функции SQL с выходными параметрами

Альтернативный способ описать результаты функции — определить её с выходными параметрами, как в этом примере:

```
CREATE FUNCTION add_em (IN x int, IN y int, OUT sum int)
AS 'SELECT x + y'
LANGUAGE SQL;
```

```
SELECT add_em(3,7);
```

add_em

10
(1 row)

Ценность выходных параметров в том, что они позволяют удобным способом определить функции, возвращающие несколько колонок. Например:

```
CREATE FUNCTION sum_n_product (x int, y int, OUT sum int, OUT product int)
AS 'SELECT x + y, x * y'
LANGUAGE SQL;
```

```
SELECT * FROM sum_n_product(11,42);
```

sum		product
-----	+	-----
53		462
(1 row)		



Функции SQL с выходными параметрами

Фактически здесь мы определили анонимный составной тип для результата функции. Показанный выше пример даёт тот же конечный результат, что и команды:

```
CREATE TYPE sum_prod AS (sum int, product int);
```

```
CREATE FUNCTION sum_n_product (int, int) RETURNS sum_prod  
AS 'SELECT $1 + $2, $1 * $2'  
LANGUAGE SQL;
```

Но предыдущий вариант зачастую удобнее - он не требует отдельно заниматься определением составного типа.

Имена, назначаемые выходным параметрам, не просто декоративные, а определяют имена колонок анонимного составного типа. (Если вы опустите имя выходного параметра, система выберет имя сама.)

Выходные параметры не включаются в список аргументов при вызове такой функции из SQL.

Это объясняется тем, что PostgreSQL определяет сигнатуру вызова функции, рассматривая только входные параметры.



Функции SQL с выходными параметрами

Это также значит, что при таких операциях, как удаление функции, в ссылках на функцию учитываются только типы входных параметров.

Таким образом, удалить эту конкретную функцию можно любой из этих команд:

```
DROP FUNCTION sum_n_product (x int, y int, OUT sum int, OUT product int);
```

```
DROP FUNCTION sum_n_product (int, int);
```

Параметры функции могут быть объявлены как IN (по умолчанию), OUT, INOUT или VARIADIC.

Параметр INOUT действует как входной (является частью списка аргументов при вызове) и как выходной (часть типа записи результата).

Параметры VARIADIC являются входными, но обрабатываются специальным образом, как описано далее.



Функции SQL с переменным числом аргументов

Функции SQL могут быть объявлены как принимающие переменное число аргументов, с условием, что все "необязательные" аргументы имеют один тип данных.

Необязательные аргументы будут переданы такой функции в виде массива. Для этого в объявлении функции последний параметр помечается как VARIADIC; при этом он должен иметь тип массива. Например:

```
CREATE FUNCTION mleast(VARIADIC arr numeric[]) RETURNS numeric AS $$  
    SELECT min($1[i]) FROM generate_subscripts($1, 1) g(i);  
$$ LANGUAGE SQL;
```

```
SELECT mleast(10, -1, 5, 4.4);
```

```
mleast
```

```
-----
```

```
-1
```

```
(1 row)
```

g(i) - псевдоним таблицы (g) и столбца (i), который возвращает generate_subscripts (генератор индексов - в данном случае возвращает набор индексов для элементов переданного массива, начиная с 1).

Выражение

FROM generate_subscripts(\$1, 1) g(i)

означает: выполнить функцию generate_subscripts и присвоить ее результат (т.е. набор индексов) таблице, которая называется "g" с одним столбцом, который называется "i"

(<https://stackoverflow.com/questions/10674735/in-postgresql-what-is-gi-in-from-generate-subscripts1-1-gi>)



Функции SQL с переменным числом аргументов

По сути, все фактические аргументы, начиная с позиции VARIADIC, собираются в одномерный массив, как если бы вы написали

```
SELECT mleast(ARRAY[10, -1, 5, 4.4]); -- это не будет работать
```

```
SELECT mleast(VARIADIC ARRAY[10, -1, 5, 4.4]); -- так работает!
```

Это предотвращает разворачивание переменного множества параметров функции в базовый тип, что позволяет сопоставить с ним значение типа массива.

VARIADIC можно добавить только к последнему фактическому аргументу вызова функции.

Также указание VARIADIC даёт единственную возможность передать пустой массив функции с переменными параметрами, например, так:

```
SELECT mleast(VARIADIC ARRAY[]::numeric[]);
```



Функции SQL с переменным числом аргументов

Простой вызов `SELECT mleast()` не будет работать, так как переменным параметрам должен соответствовать минимум один фактический аргумент. (Можно определить вторую функцию с таким же именем `mleast`, но без параметров, если вы хотите выполнять такие вызовы.)

Элементы массива, создаваемые из переменных параметров, считаются не имеющими собственных имён.

Это означает, что передать функции с переменными параметрами именованные аргументы нельзя, если только при вызове не добавлено `VARIADIC`. Например, этот вариант будет работать:

```
SELECT mleast(VARIADIC arr := ARRAY[10, -1, 5, 4.4]);
```

А эти варианты нет:

```
SELECT mleast(arr := 10);
```

```
SELECT mleast(arr := ARRAY[10, -1, 5, 4.4]);
```



Функции SQL, порождающие таблицы

Все функции SQL можно использовать в предложении FROM запросов, но наиболее полезно это для функций, возвращающих составные типы.

Если функция объявлена как возвращающая базовый тип, она возвращает таблицу с одной колонкой.

Если же функция объявлена как возвращающая составной тип, она возвращает таблицу с колонками для каждого атрибута составного типа.

Например:

```
CREATE TABLE foo (fooid int, foosubid int, fooname text);  
INSERT INTO foo VALUES (1, 1, 'Joe');  
INSERT INTO foo VALUES (1, 2, 'Ed');  
INSERT INTO foo VALUES (2, 1, 'Mary');
```

```
CREATE FUNCTION getfoo(int) RETURNS foo AS $$  
    SELECT * FROM foo WHERE fooid = $1;  
$$ LANGUAGE SQL;
```



Функции SQL, порождающие таблицы

```
SELECT *, upper(foaname) FROM getfoo(1) AS t1;
```

```
fooid | foosubid | foaname | upper
```

```
-----+-----+-----+-----
```

```
1 | 1 | Joe | JOE
```

```
(1 row)
```

Как показывает этот пример, мы можем работать с колонками результата функции так же, как если бы это были колонки обычной таблицы.

Заметьте, что мы получаем из данной функции только одну строку. Это объясняется тем, что мы не использовали указание SETOF (см. далее).



Функции SQL, возвращающие множества

Когда SQL-функция объявляется как возвращающая SETOF некий_тип, конечный запрос функции выполняется до завершения и каждая строка выводится как элемент результирующего множества.

Это обычно используется, когда функция вызывается в предложении FROM. В этом случае каждая строка, возвращаемая функцией, становится строкой таблицы, появляющейся в запросе. Например, в предположении, что таблица foo имеет то же содержимое, что и раньше, мы выполняем:

```
CREATE FUNCTION getfoo(int) RETURNS SETOF foo AS $$  
  SELECT * FROM foo WHERE fooid = $1;  
$$ LANGUAGE SQL;  
SELECT * FROM getfoo(1) AS t1;
```

Тогда в ответе мы получим все строки результата:

fooid	foosubid	fooname
1	1	Joe
1	2	Ed

(2 rows)



Функции SQL, возвращающие множества

Также возможно выдать несколько строк с колонками, определяемыми выходными параметрами, следующим образом:

```
CREATE TABLE tab (y int, z int);  
INSERT INTO tab VALUES (1, 2), (3, 4), (5, 6), (7, 8);
```

```
CREATE FUNCTION sum_n_product_with_tab (x int, OUT sum int, OUT product int)  
RETURNS SETOF record  
AS $$  
    SELECT $1 + tab.y, $1 * tab.y FROM tab;  
$$ LANGUAGE SQL;
```

```
SELECT * FROM sum_n_product_with_tab(10);
```

```
sum | product  
-----+-----  
11 |      10  
13 |      30  
15 |      50  
17 |      70  
(4 rows)
```

Здесь ключевая особенность заключается в записи RETURNS SETOF record, показывающей, что функция возвращает множество строк вместо одной. Если существует только один выходной параметр, укажите тип этого параметра вместо record.



Функции SQL, возвращающие множества

Часто бывает полезно сконструировать результат запроса, вызывая функцию, возвращающую множество, несколько раз, передавая при каждом вызове параметры из очередных строк таблицы или подзапроса.

Для этого рекомендуется применить ключевое слово LATERAL.

Ниже приведён пример использования функции, возвращающей множество, для перечисления элементов древовидной структуры:

```
SELECT * FROM nodes;
```

name		parent
Top		
Child1		Top
Child2		Top
Child3		Top
SubChild1		Child1
SubChild2		Child1

(6 rows)

```
CREATE FUNCTION  
listchildren(text) RETURNS SETOF  
text AS $$  
    SELECT name FROM nodes  
    WHERE parent = $1  
$$ LANGUAGE SQL STABLE;
```

```
SELECT * FROM listchildren('Top');  
listchildren  
-----  
Child1  
Child2  
Child3  
(3 rows)
```



Функции SQL, возвращающие множества

```
SELECT name, child FROM nodes, LATERAL listchildren(name)  
AS child;
```

name		child
Top		Child1
Top		Child2
Top		Child3
Child1		SubChild1
Child1		SubChild2

(5 rows)



Функции SQL, возвращающие множества

В настоящее время функции, возвращающие множества, могут также вызываться в списке выборки запроса.

Для каждой строки, которая генерируется самим запросом, вызывается функция, возвращающая множество, и для каждого элемента набора её результатов генерируется отдельная строка.

Заметьте, однако, что эта возможность считается устаревшей и может быть ликвидирована в будущих выпусках. Предыдущий пример можно было бы также переписать с применением запросов следующим образом:

SELECT listchildren('Top');

listchildren

Child1

Child2

Child3

(3 rows)

SELECT name, listchildren(name) FROM nodes;

name | listchildren

-----+-----

Top | Child1

Top | Child2

Top | Child3

Child1 | SubChild1

Child1 | SubChild2

(5 rows)

В данной команде SELECT для Child2, Child3 и т. д. строки не выдаются. Это происходит потому, что listchildren возвращает пустое множество для этих аргументов, так что строки результата не генерируются. Это же поведение мы получаем при внутреннем соединении с результатом функции с применением LATERAL.



Функции SQL, возвращающие таблицы (TABLE)

Есть ещё один способ объявить функцию, возвращающую множества, — использовать синтаксис RETURNS TABLE(колонки). Это равнозначно использованию одного или нескольких параметров OUT с объявлением функции как возвращающей SETOF record (или SETOF тип единственного параметра, если это применимо). Этот синтаксис описан в последних версиях стандарта SQL, так что этот вариант может быть более портируемым, чем SETOF.

Например, предыдущий пример с суммой и произведением можно также переписать так:

```
CREATE FUNCTION sum_n_product_with_tab (x int)
RETURNS TABLE(sum int, product int) AS $$
    SELECT $1 + tab.y, $1 * tab.y FROM tab;
$$ LANGUAGE SQL;
```

Запись RETURNS TABLE не позволяет явно указывать OUT и INOUT для параметров — все выходные колонки необходимо записать в списке TABLE.



Полиморфные типы

Особый интерес представляют пять псевдотипов: `anyelement`, `anyarray`, `anyonarray`, `anyenum` и `anyrange`, которые называются полиморфными типами. Функция, в объявлении которой используются эти типы, называется полиморфной. Полиморфная функция может работать со множеством различных типов данных; конкретный тип определяется в зависимости от значения, переданного при вызове.

В каждой позиции (в аргументах или возвращаемом значении), объявленной как `anyelement`, может передаваться любой фактический тип данных, но в каждом конкретном вызове все эти фактические типы должны быть одинаковыми.

Аналогичным образом, в каждой позиции, объявленной как `anyarray`, может передаваться любой тип данных массива, но все фактические типы массивов должны совпадать.



Полиморфные типы

Так же и во всех позициях, объявленных как `anyrange`, должен передаваться одинаковый тип-диапазон.

Более того, если некоторые позиции объявлены как `anyarray`, а другие как `anyelement`, то фактическим типом в позициях `anyarray` должен быть массив, элементы которого имеют тот же тип, что и значения в позициях `anyelement`.

Подобным образом, если одни позиции объявлены как `anyrange`, а другие как `anyelement`, фактическим типом в позициях `anyrange` должен быть диапазон, подтип которого совпадает с типом, передаваемым в позициях `anyelement`.

Псевдотип `anynonarray` обрабатывается так же, как `anyelement`, но с дополнительным ограничением — фактический тип не должен быть типом массива.

Псевдотип `anyenum` тоже обрабатывается как `anyelement`, но его фактические типы ограничиваются перечислениями.



Полиморфные функции SQL

Функции SQL могут быть объявлены как принимающие и возвращающие полиморфные типы `anyelement`, `anyarray`, `anynonarray`, `anyenum` и `anyrange`. В следующем примере полиморфная функция `make_array` создаёт массив из двух элементов произвольных типов:

```
CREATE FUNCTION make_array(anyelement, anyelement) RETURNS anyarray AS $$  
    SELECT ARRAY[$1, $2];  
$$ LANGUAGE SQL;
```

```
SELECT make_array(1, 2) AS intarray, make_array('a'::text, 'b') AS textarray;
```

```
intarray | textarray
```

```
-----+-----
```

```
{1,2}   | {a,b}
```

```
(1 row)
```

Обратите внимание на приведение типа `'a'::text`, определяющее, что аргумент имеет тип `text`. Оно необходимо, если аргумент задаётся просто строковой константой, так как иначе он будет воспринят как имеющий тип `unknown`, а массив типов `unknown` является недопустимым. Без этого приведения вы получите такую ошибку:

ERROR: could not determine polymorphic type because input has type "unknown"



Полиморфные функции SQL

Функция с полиморфными аргументами может иметь фиксированный тип результата, однако обратное не допускается. Например:

```
CREATE FUNCTION is_greater(anyelement, anyelement) RETURNS boolean AS $$
```

```
    SELECT $1 > $2;
```

```
$$ LANGUAGE SQL;
```

```
SELECT is_greater(1, 2);
```

```
is_greater
```

```
-----
```

```
f
```

```
(1 row)
```

```
CREATE FUNCTION invalid_func() RETURNS anyelement AS $$
```

```
    SELECT 1;
```

```
$$ LANGUAGE SQL;
```

```
ERROR: cannot determine result data type
```

```
DETAIL: A function returning a polymorphic type must have at  
least one polymorphic argument.
```

(ОШИБКА: не удалось определить тип результата;
ПОДРОБНОСТИ: Функция, возвращающая полиморфный тип,
обязана иметь минимум один полиморфный аргумент.)



Полиморфные функции SQL

Полиморфизм можно применять и с функциями, имеющими выходные аргументы.
Например:

```
CREATE FUNCTION dup (f1 anyelement, OUT f2 anyelement, OUT f3 anyarray)  
AS 'select $1, array[$1,$1]' LANGUAGE SQL;
```

```
SELECT * FROM dup(22);
```

```
f2 | f3
```

```
-----+-----
```

```
22 | {22,22}
```

```
(1 row)
```



Полиморфные функции SQL

Полиморфизм также можно применять с функциями с переменными параметрами. Например:

```
CREATE FUNCTION anyleast (VARIADIC anyarray)
RETURNS anyelement AS $$
```

```
    SELECT min($1[i]) FROM generate_subscripts($1, 1)
    g(i);
$$ LANGUAGE SQL;
```

```
SELECT anyleast(10, -1, 5, 4);
```

anyleast

-1

(1 row)

```
SELECT anyleast('abc'::text, 'def');
```

anyleast

abc

(1 row)

```
CREATE FUNCTION concat_values(text,
VARIADIC anyarray) RETURNS text AS $$
```

```
    SELECT array_to_string($2, $1);
```

```
$$ LANGUAGE SQL;
```

```
SELECT concat_values('|', 1, 4, 2);
```

concat_values

1|4|2

(1 row)



Перегрузка функций

Вы можете определить несколько функций с одним именем SQL, если эти функции будут принимать разные аргументы.

Другими словами, имена функций можно перегружать.

Когда выполняется запрос, сервер определяет, какую именно функцию вызывать, по количеству и типам представленных аргументов.

Перегрузка может быть полезна для имитации функций с переменным количеством аргументов, до какого-то конечного числа.

Создавая семейство перегруженных функций, необходимо не допускать неоднозначности. Например, если созданы функции:

```
CREATE FUNCTION test(int, real) RETURNS ...
```

```
CREATE FUNCTION test(smallint, double precision) RETURNS ...
```

не вполне понятно, какая функция будет вызвана с довольно простыми аргументами вроде `test(1, 1.5)`.



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Подпрограммы



Функции

- вызываются в контексте выражения
- не могут управлять транзакциями
- возвращают результат

Процедуры

- вызываются оператором CALL
- могут управлять транзакциями
- могут возвращать результат



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Процедуры были введены в PostgreSQL 11. Основная причина их появления состоит в том, что функции не могут управлять транзакциями. Функции вызываются в контексте какого-либо выражения, которое вычисляется как часть уже начатого оператора (например, SELECT) в уже начатой транзакции. Нельзя завершить транзакцию и начать новую «посередине» выполнения оператора.

Процедуры всегда вызываются специальным оператором CALL. Если этот оператор сам начинает новую транзакцию (а не вызывается из уже начатой), то в процедуре можно использовать команды управления транзакциями.

К сожалению, процедуры, написанные на языке SQL, лишены сейчас возможности использовать команды COMMIT и ROLLBACK, хотя те из них, что оформлены в новом стиле стандарта SQL, вероятно смогут это делать в будущем. Поэтому пример процедуры, управляющей транзакциями, придется отложить до темы «PL/pgSQL. Выполнение запросов».



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Иногда можно услышать, что процедура отличается от функции тем, что не возвращает результат. Но это не так — процедуры тоже могут возвращать результат, если необходимо.

Функции и процедуры имеют общее пространство имен и вместе называются *подпрограммами (routine)*.

<https://postgrespro.ru/docs/postgresql/16/sql-createprocedure>

<https://postgrespro.ru/docs/postgresql/16/sql-call>



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Процедуры без параметров

Начнем с примера простой процедуры без параметров.

```
=> CREATE TABLE t(a float);
```

```
CREATE TABLE
```

```
=> CREATE PROCEDURE fill()
```

```
AS $$
```

```
    TRUNCATE t;
```

```
    INSERT INTO t SELECT random() FROM generate_series(1,3);
```

```
$$ LANGUAGE sql;
```

```
CREATE PROCEDURE
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Чтобы вызвать процедуру, необходимо использовать специальный оператор:

```
=> CALL fill();
```

CALL

Результат работы виден в таблице:

```
=> SELECT * FROM t;
```

a
0.7421375686259004
0.9144090559017666
0.2113045363655326

(3 rows)



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

А теперь переопределим нашу процедуру в стиле стандарта SQL:

```
=> CREATE OR REPLACE PROCEDURE fill()
```

```
LANGUAGE sql
```

```
BEGIN ATOMIC
```

```
    DELETE FROM t; -- команда TRUNCATE пока что не поддерживается в таких подпрограммах
```

```
    INSERT INTO t SELECT random() FROM generate_series(1,3);
```

```
END;
```

```
CREATE PROCEDURE
```

Убедимся в ее работоспособности:

```
=> CALL fill();
```

```
CALL
```

```
=> SELECT * FROM t;
```

```
      a
```

```
-----
```

```
0.962614132914897
```

```
0.3921016261409911
```

```
0.8097851961481584
```

```
(3 rows)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:
https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Процедуры с параметрами

Добавим в процедуру входной параметр — число строк:

```
=> DROP PROCEDURE fill();
```

```
DROP PROCEDURE
```

```
=> CREATE PROCEDURE fill(nrows integer)
LANGUAGE sql
BEGIN ATOMIC
    DELETE FROM t;
    INSERT INTO t SELECT random() FROM generate_series(1, nrows);
END;
```

```
CREATE PROCEDURE
```

Точно так же, как и в случае функций, при вызове процедур фактические параметры можно передавать позиционным способом или по имени:

```
=> CALL fill(nrows => 5);
```

```
CALL
```

```
=> SELECT * FROM t;
```

```
          a
-----
 0.6171391710234337
 0.7274933209643315
 0.7617533228462221
 0.08398194726898378
 0.7356269859122342
(5 rows)
```



Следующий материал взят из учебного курса DEV1 Postgres Pro:

https://edu.postgrespro.ru/16/dev1-16/dev1_09_sql_proc.html

Процедуры могут также иметь OUT- и INOUT-параметры, с помощью которых можно возвращать значения:

```
=> DROP PROCEDURE fill(integer);
```

DROP PROCEDURE

```
=> CREATE PROCEDURE fill(IN nrows integer, OUT average float)
LANGUAGE sql
BEGIN ATOMIC
    DELETE FROM t;
    INSERT INTO t SELECT random() FROM generate_series(1, nrows);
    SELECT avg(a) FROM t; -- как в функции
END;
```

CREATE PROCEDURE

Попробуем:

```
=> CALL fill(5, NULL /* значение не используется, но его необходимо указать*/);
```

```
         average
-----
0.5622983543902438
(1 row)
```



Язык PL/pgSQL



Структура PL/pgSQL

Функции, написанные на PL/pgSQL, определяются на сервере командами CREATE FUNCTION. Такая команда обычно выглядит, например, так:

```
CREATE FUNCTION somefunc(integer, text) RETURNS integer
```

```
AS 'тело функции'
```

```
LANGUAGE plpgsql;
```

Если рассматривать CREATE FUNCTION, тело функции представляет собой просто текстовую строку.

Часто для написания тела функции удобнее заключать эту строку в доллары, а не в обычные апострофы. Если не применять заключение в доллары, все апострофы или обратные косые черты в теле функции придётся экранировать, дублируя их.



Структура PL/pgSQL

PL/pgSQL - это блочно-структурированный язык. Текст тела функции должен быть блоком. Структура блока:

[<<метка>>]

[DECLARE

 объявления]

BEGIN

 операторы

END [метка];

Каждое объявление и каждый оператор в блоке должны завершаться символом «;».

Блок, вложенный в другой блок, должен иметь точку с запятой после END, как показано выше. Однако финальный END, завершающий тело функции, не требует точки с запятой.

Распространённой ошибкой является добавление точки с запятой сразу после BEGIN. Это неправильно и приведёт к синтаксической ошибке.

Метка требуется только тогда, когда нужно идентифицировать блок в операторе EXIT, или дополнить имена переменных, объявленных в этом блоке.

Если метка указана после END, то она должна совпадать с меткой в начале блока.



Структура PL/pgSQL

Во время выполнения вложенного блока переменные, объявленные в нём, скрывают переменные внешних блоков с такими же именами. Чтобы получить доступ к внешним переменным, нужно дополнить их имена меткой блока. Например:

```
CREATE FUNCTION somefunc() RETURNS integer AS $$
<< outerblock >>
DECLARE
    quantity integer := 30;
BEGIN
    RAISE NOTICE 'Сейчас quantity = %', quantity; -- Выводится 30
    quantity := 50;
    -- Вложенный блок
    DECLARE
        quantity integer := 80;
    BEGIN
        RAISE NOTICE 'Сейчас quantity = %', quantity; -- Выводится 80
        RAISE NOTICE 'Во внешнем блоке quantity = %', outerblock.quantity; -- Выводится 50
    END;
    RAISE NOTICE 'Сейчас quantity = %', quantity; -- Выводится 50
    RETURN quantity;
END;
$$ LANGUAGE plpgsql;
```



Объявления параметров функции

Переданные в функцию параметры именуются идентификаторами \$1, \$2 и т. д. Дополнительно, для улучшения читаемости, можно объявить псевдонимы для параметров \$n. Либо псевдоним, либо цифровой идентификатор используются для обозначения параметра.

Создать псевдоним можно двумя способами. Предпочтительный способ это дать имя параметру в команде CREATE FUNCTION, например:

```
CREATE FUNCTION sales_tax(subtotal real) RETURNS real AS $$  
BEGIN  
    RETURN subtotal * 0.06;  
END;  
$$ LANGUAGE plpgsql;
```



Объявления параметров функции

Другой способ это явное объявление псевдонима при помощи синтаксиса:

имя ALIAS FOR \$n;

Предыдущий пример для этого стиля выглядит так:

```
CREATE FUNCTION sales_tax(real) RETURNS real AS $$
```

```
DECLARE
```

```
    subtotal ALIAS FOR $1;
```

```
BEGIN
```

```
    RETURN subtotal * 0.06;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Эти два примера не полностью эквивалентны. В первом случае на subtotal можно ссылаться как sales_tax.subtotal, а во втором случае такая ссылка невозможна. (Если бы к внутреннему блоку была добавлена метка, то subtotal можно было бы дополнить этой меткой.)



Объявления параметров функции

Другие примеры:

```
CREATE FUNCTION instr(vchar, integer) RETURNS integer AS $$  
DECLARE  
    v_string ALIAS FOR $1;  
    index ALIAS FOR $2;  
BEGIN  
    -- вычисления, использующие v_string и index  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE FUNCTION concat_selected_fields(in_t sometablename) RETURNS text AS $$  
BEGIN  
    RETURN in_t.f1 || in_t.f3 || in_t.f5 || in_t.f7;  
END;  
$$ LANGUAGE plpgsql;
```



Объявления параметров функции

Выходные параметры наиболее полезны для возвращения нескольких значений.
Тривиальный пример:

```
CREATE FUNCTION sum_n_product(x int, y int, OUT sum int, OUT prod int) AS $$  
BEGIN  
    sum := x + y;  
    prod := x * y;  
END;  
$$ LANGUAGE plpgsql;
```

```
SELECT * FROM sum_n_product(2, 4);  
sum | prod  
-----+-----  
6 | 8
```

Здесь фактически создаётся анонимный тип record для возвращения результата функции. Если предложение RETURNS используется, то оно обязано выглядеть как RETURNS record.



Объявления параметров функции

Это также работает с процедурами, например:

```
CREATE PROCEDURE sum_n_product(x int, y int, OUT sum int, OUT prod int) AS $$  
BEGIN  
    sum := x + y;  
    prod := x * y;  
END;  
$$ LANGUAGE plpgsql;
```

В вызове процедуры должны быть указаны все параметры. Для выходных параметров может указываться NULL в вызове:

```
CALL sum_n_product(2, 4, NULL, NULL);  
sum | prod  
-----+-----  
6   |    8
```

Однако когда вызов процедуры происходит из PL/pgSQL, необходимо писать переменную для каждого выходного параметра; в переменную запишется результат вызова.



Объявления параметров функции

Для функции на PL/pgSQL, возвращающей полиморфный тип (anyelement, anyarray, anyonarray, anyenum, anyrange), создаётся специальный параметр \$0.

Его тип данных соответствует типу, фактически возвращаемому функцией, и который устанавливается на основании фактических типов входных параметров.

Это позволяет функции обращаться к фактически возвращаемому типу данных.

Параметр \$0 инициализируется в NULL и его можно изменять внутри функции.

Таким образом, его можно использовать для хранения возвращаемого значения, хотя это необязательно.

Параметру \$0 можно дать псевдоним.



Основные операторы. Выполнение SQL-команды (не возвращает результат)

В функции на PL/pgSQL можно выполнить любую команду SQL, не возвращающую строк, просто написав эту команду (например, INSERT без предложения RETURNING).

Имя любой переменной PL/pgSQL в тексте команды рассматривается как параметр, а затем текущее значение переменной подставляется в качестве значения параметра во время выполнения.

При выполнении SQL-команды таким образом, PL/pgSQL может кешировать и повторно использовать план выполнения команды.

Иногда бывает полезно вычислить значение выражения или запроса SELECT, но отказаться от результата, например, при вызове функции, у которой есть побочные эффекты, но нет полезного результата. Для этого в PL/pgSQL, используется оператор PERFORM:

PERFORM запрос;

Эта команда выполняет запрос и отбрасывает результат.



Основные операторы. Выполнение SQL-команды (не возвращает результат)

Запросы пишутся таким же образом, как и в команде SQL SELECT, но ключевое слово SELECT заменяется на PERFORM.

Для запросов WITH после PERFORM нужно поместить запрос в скобки. (В этом случае запрос может вернуть только одну строку.)

Переменные PL/pgSQL будут подставлены в запрос так же, как и в команду, не возвращающую результат.

Кроме того, специальная переменная FOUND устанавливается в истину, если запрос возвращает, по крайней мере, одну строку, или ложь, если не возвращает ни одной строки.

Пример:

```
PERFORM create_mv('cs_session_page_requests_mv', my_query);
```



Основные операторы. Выполнение запроса, возвращающего одну строку

Результат SQL-команды, возвращающей одну строку (возможно из нескольких столбцов), может быть присвоен переменной типа record, переменной строкового типа или списку скалярных переменных. Для этого нужно к основной команде SQL добавить предложение INTO. Так, например:

```
SELECT выражения_select INTO [STRICT] цель FROM ...;
```

```
INSERT ... RETURNING выражения INTO [STRICT] цель;
```

```
UPDATE ... RETURNING выражения INTO [STRICT] цель;
```

```
DELETE ... RETURNING выражения INTO [STRICT] цель;
```

где цель может быть переменной типа record, строковой переменной или разделённым запятыми списком скалярных переменных, полей записи/строки.



Основные операторы. Выполнение динамически формируемых команд

Это команды, в которых при каждом выполнении могут использоваться разные таблицы или типы данных.

Обычно PL/pgSQL кеширует планы выполнения, но в случае с динамическими командами это не будет работать. Для исполнения динамических команд предусмотрен оператор EXECUTE:

EXECUTE строка-команды [INTO [STRICT] цель] [USING выражение [, ...]];

где строка-команды это выражение, формирующее строку (типа text) с текстом команды, которую нужно выполнить.

Необязательная цель — это переменная-запись, переменная-кортеж или разделённый запятыми список простых переменных и полей записи/кортежа, куда будут помещены результаты команды.

Необязательные выражения в USING формируют значения, которые будут вставлены в команду.



Основные операторы. Выполнение динамически формируемых команд

С указанием STRICT запрос должен вернуть ровно одну строку, иначе выдаётся сообщение об ошибке.

В тексте команды можно использовать значения параметров, ссылки на параметры обозначаются как \$1, \$2 и т. д. Эти символы указывают на значения, находящиеся в предложении USING. Пример:

```
EXECUTE 'SELECT count(*) FROM mytable WHERE inserted_by = $1 AND inserted <= $2'  
  
INTO c  
  
USING checked_user, checked_date;
```



Основные операторы. Выполнение динамически формируемых команд

Обратите внимание, что символы параметров можно использовать только вместо значений данных.

Если же требуется динамически формировать имена таблиц или столбцов, их необходимо вставлять в виде текста.

Например, если в предыдущем запросе необходимо динамически задавать имя таблицы, можно сделать следующее:

```
EXECUTE 'SELECT count(*) FROM ' -- части команды конкатенируются с вызовом функции
```

```
|| quote_ident(tabname) --tabname - в качестве идентификатора в SQL -операторе, при необх. закл-ся в кавычки
```

```
|| ' WHERE inserted_by = $1 AND inserted <= $2'
```

```
INTO c
```

```
USING checked_user, checked_date;
```



Основные операторы. Выполнение динамически формируемых команд

В качестве более аккуратного решения, вместо имени таблиц или столбцов можно использовать указание формата %I с функцией format() (текст, разделённый символами новой строки, соединяется вместе):

```
EXECUTE format('SELECT count(*) FROM %I '  
              'WHERE inserted_by = $1 AND inserted <= $2', tablename)  
  
INTO c  
  
USING checked_user, checked_date;
```

Ещё одно ограничение состоит в том, что символы параметров могут использоваться только в командах SELECT, INSERT, UPDATE и DELETE.

В операторы других типов (обычно называемые служебными) значения нужно вставлять в текстовом виде, даже если это просто значения данных.



Основные операторы. Выполнение динамически формируемых команд

Динамические операторы SQL также можно безопасно сформировать, используя функцию `format`.

```
EXECUTE format('UPDATE tbl SET %I = %L '  
              'WHERE key = %L', colname, newvalue, keyvalue);
```

Указание `%I` равнозначно вызову `quote_ident`, а `%L` — вызову `quote_nullable`.
Функция `format` может применяться в сочетании с предложением `USING`:

```
EXECUTE format('UPDATE tbl SET %I = $1 WHERE key = $2', colname)  
          USING newvalue, keyvalue;
```

Эта форма лучше, так как с ней переменные обрабатываются в их собственном формате данных, а не преобразуются безусловно в текст, чтобы затем выводиться с использованием `%L`. Она также и более эффективна.



Основные операторы. Присваивание

Если тип данных результата выражения не соответствует типу данных переменной, это значение будет преобразовано к нужному типу с использованием приведения присваивания.

В случае отсутствия приведения присваивания для этой пары типов, интерпретатор PL/pgSQL попытается преобразовать значение результата через текстовый формат, то есть применив функцию вывода типа результата, а за ней функцию ввода типа переменной.

Заметьте, что при этом функция ввода может выдавать ошибки времени выполнения, если не воспримет строковое представление значения результата.

Примеры:

```
tax := subtotal * 0.06;  
my_record.user_id := 20;  
my_array[j] := 20;  
my_array[1:3] := array[1,2,3];  
complex_array[n].realpart = 12.3;
```



Управляющие структуры. Возврат значений

Две команды позволяют вернуть данные из функции: RETURN и RETURN NEXT.

RETURN выражение;

RETURN с последующим выражением прекращает выполнение функции и возвращает значение выражения в вызывающую программу. Эта форма используется для функций PL/pgSQL, которые не возвращают набор строк.

В функции, возвращающей скалярный тип, результирующее выражение автоматически приводится к типу возвращаемого значения. Однако если возвращаемый тип — составной (строка), возвращаемое выражение должно в точности содержать требуемый набор столбцов. При этом может потребоваться явное приведение типов.

Для функции с выходными параметрами просто используйте RETURN без выражения. Будут возвращены текущие значения выходных параметров.

Для функции, возвращающей void, RETURN можно использовать в любом месте, но без выражения после RETURN.



Управляющие структуры. RETURN NEXT и RETURN QUERY

RETURN NEXT выражение;

RETURN QUERY запрос;

RETURN QUERY EXECUTE строка-команды [USING выражение [, ...]];

Для функций на PL/pgSQL, возвращающих SETOF некий_тип, нужно использовать RETURN NEXT или RETURN QUERY. Финальная команда RETURN без аргументов завершает выполнение функции.

RETURN NEXT используется как со скалярными, так и с составными типами данных.

Для составного типа результат функции возвращается в виде таблицы.

RETURN QUERY добавляет результат выполнения запроса к результату функции.

RETURN NEXT и RETURN QUERY можно свободно смешивать в теле функции, в этом случае их результаты будут объединены.



Управляющие структуры. RETURN NEXT и RETURN QUERY

Для функции с выходными параметрами просто используйте RETURN NEXT без аргументов.

При каждом исполнении RETURN NEXT текущие значения выходных параметров сохраняются для последующего возврата в качестве строки результата.

Обратите внимание, что если функция с выходными параметрами должна возвращать множество значений, то при объявлении нужно указывать RETURNS SETOF.

При этом если выходных параметров несколько, то используется RETURNS SETOF record, а если только один с типом некий_тип, то RETURNS SETOF некий_тип.



Управляющие структуры. If - then

```
IF логическое-выражение THEN  
    операторы  
END IF;
```

IF-THEN это простейшая форма IF. Операторы между THEN и END IF выполняются, если условие (логическое-выражение) истинно. В противном случае они опускаются.

Пример:

```
IF v_user_id <> 0 THEN  
    UPDATE users SET email = v_email WHERE user_id = v_user_id;  
END IF;
```



Управляющие структуры. If – then - elsif

IF логическое-выражение THEN

операторы

[ELSIF логическое-выражение THEN операторы [ELSIF логическое-выражение THEN операторы ...]]

[ELSE операторы]

END IF;

IF-THEN-ELSIF обеспечивает удобный способ проверки нескольких вариантов по очереди.

Условия в IF последовательно проверяются до тех пор, пока не будет найдено первое истинное. После этого операторы, относящиеся к этому условию, выполняются, и управление переходит к следующей после END IF команде.

Если ни одно из условий IF не является истинным, то выполняется блок ELSE (если присутствует).



Управляющие структуры. Простой CASE

CASE выражение-поиска

WHEN выражение [, выражение [...]] THEN

операторы

[WHEN выражение [, выражение [...]] THEN операторы ...]

[ELSE операторы]

END CASE;

Простая форма CASE реализует условное выполнение на основе сравнения операндов.

Выражение-поиска вычисляется (один раз) и последовательно сравнивается с каждым выражением в условиях WHEN.

Если совпадение найдено, то выполняются соответствующие операторы и управление переходит к следующей после END CASE команде. (Все последующие выражения WHEN не проверяются.)

Если совпадение не было найдено, то выполняются операторы в ELSE. Но если ELSE нет, то вызывается исключение CASE_NOT_FOUND.



Управляющие структуры. CASE с перебором условий

CASE

WHEN логическое-выражение THEN
операторы

[WHEN логическое-выражение THEN операторы ...]

[ELSE операторы]

END CASE;

Эта форма CASE реализует условное выполнение, основываясь на истинности логических условий.

Каждое логическое-выражение в предложении WHEN вычисляется по порядку до тех пор, пока не будет найдено истинное.

Затем выполняются соответствующие операторы и управление переходит к следующей после END CASE команде. (Все последующие выражения WHEN не проверяются.)

Если ни одно из условий не окажется истинным, то выполняются операторы в ELSE. Но если ELSE нет, то вызывается исключение CASE_NOT_FOUND.



Управляющие структуры. Простые циклы. LOOP

Операторы LOOP, EXIT, CONTINUE, WHILE, FOR и FOREACH позволяют повторить серию команд в функции на PL/pgSQL.

```
[<<метка>>]
```

```
LOOP
```

```
    операторы
```

```
END LOOP [ метка ];
```

LOOP организует безусловный цикл, который повторяется до бесконечности, пока не будет прекращён операторами EXIT или RETURN.

Для вложенных циклов можно использовать метку в операторах EXIT и CONTINUE, чтобы указать, к какому циклу эти операторы относятся.



Управляющие структуры. Простые циклы. EXIT

EXIT [метка] [WHEN логическое-выражение];

Если метка не указана, то завершается самый внутренний цикл, далее выполняется оператор, следующий за END LOOP.

Если метка указана, то она должна относиться к текущему или внешнему циклу, или это может быть метка блока.

При этом в именованном цикле/блоке выполнение прекращается, а управление переходит к следующему оператору после соответствующего END.

При наличии WHEN цикл прекращается, только если логическое-выражение истинно. В противном случае управление переходит к оператору, следующему за EXIT.

EXIT можно использовать со всеми типами циклов, не только с безусловным.

Когда EXIT используется для выхода из блока, управление переходит к следующему оператору после окончания блока. Обратите внимание, что для выхода из блока нужно обязательно указывать метку. EXIT без метки не позволяет прекратить работу блока. (Это изменение по сравнению с версиями PostgreSQL до 8.4, в которых разрешалось использовать EXIT без метки для прекращения работы текущего блока.)



Управляющие структуры. Простые циклы. EXIT

Примеры:

LOOP

-- здесь производятся вычисления

IF count > 0 THEN

EXIT; -- выход из цикла

END IF;

END LOOP;

LOOP

-- здесь производятся вычисления

EXIT WHEN count > 0; -- аналогично предыдущему примеру

END LOOP;

<<ablock>>

BEGIN

-- здесь производятся вычисления

IF stocks > 100000 THEN

EXIT ablock; -- выход из блока BEGIN

END IF;

-- вычисления не будут выполнены, если stocks > 100000

END;



Управляющие структуры. CONTINUE

CONTINUE [метка] [WHEN логическое-выражение];

Если метка не указана, то начинается следующая итерация самого внутреннего цикла. То есть все оставшиеся в цикле операторы пропускаются, и управление переходит к управляющему выражению цикла (если есть) для определения, нужна ли ещё одна итерация цикла.

Если метка присутствует, то она указывает на метку цикла, выполнение которого будет продолжено. При наличии WHEN следующая итерация цикла начинается только тогда, когда логическое-выражение истинно. В противном случае управление переходит к оператору, следующему за CONTINUE. CONTINUE можно использовать со всеми типами циклов, не только с безусловным.

Примеры:

LOOP

-- здесь производятся вычисления

EXIT WHEN count > 100;

CONTINUE WHEN count < 50;

-- вычисления для count в диапазоне 50 .. 100

END LOOP;



Управляющие структуры. WHILE

[<<метка>>]

WHILE логическое-выражение LOOP

операторы

END LOOP [метка];

WHILE выполняет серию команд до тех пор, пока истинно логическое-выражение. Выражение проверяется непосредственно перед каждым входом в тело цикла.

Пример:

WHILE amount_owed > 0 AND gift_certificate_balance > 0 LOOP

-- здесь производятся вычисления

END LOOP;

WHILE NOT done LOOP

-- здесь производятся вычисления

END LOOP;



Управляющие структуры.

FOR (целочисленный вариант)

[<<метка>>]

FOR имя IN [REVERSE] выражение .. выражение [BY выражение] LOOP

операторы

END LOOP [метка];

В этой форме цикла FOR итерации выполняются по диапазону целых чисел.

Переменная имя автоматически определяется с типом integer и существует только внутри цикла (если уже существует переменная с таким именем, то внутри цикла она будет игнорироваться).

Выражения для нижней и верхней границы диапазона чисел вычисляются один раз при входе в цикл.

Если не указано BY, то шаг итерации 1, в противном случае используется значение в BY, которое вычисляется, опять же, один раз при входе в цикл.

Если указано REVERSE, то после каждой итерации величина шага вычитается, а не добавляется.



Управляющие структуры. Цикл по результатам запроса

Другой вариант FOR позволяет организовать цикл по результатам запроса. Синтаксис:

```
[ <<метка>> ]  
FOR цель IN запрос LOOP  
    операторы  
END LOOP [ метка ];
```

Переменная цель может быть строковой переменной, переменной типа record или разделённым запятыми списком скалярных переменных.

Переменной *цель* последовательно присваиваются строки результата запроса, и для каждой строки выполняется тело цикла.



Управляющие структуры. Цикл по элементам массива

Цикл FOREACH очень похож на FOR.

Отличие в том, что вместо перебора строк SQL-запроса происходит перебор элементов массива. (В целом, FOREACH предназначен для перебора выражений составного типа. Варианты реализации цикла для работы с прочими составными выражениями помимо массивов могут быть добавлены в будущем.)

Синтаксис цикла FOREACH:

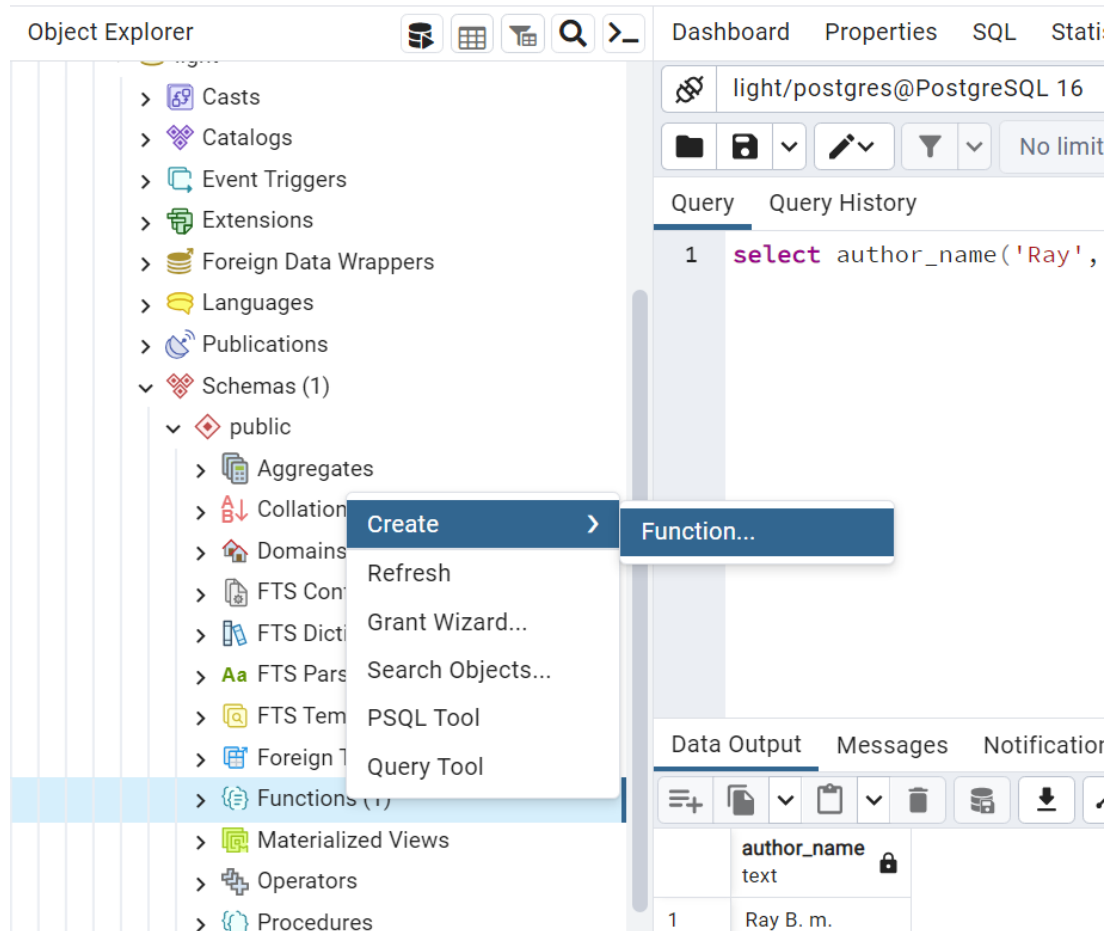
```
[ <<метка>> ]  
FOREACH цель [ SLICE число ] IN ARRAY выражение LOOP  
    операторы  
END LOOP [ метка ];
```

Без указания SLICE, или если SLICE равен 0, цикл выполняется по всем элементам массива, полученного из выражения.

Переменной **цель** последовательно присваивается каждый элемент массива и для него выполняется тело цикла.



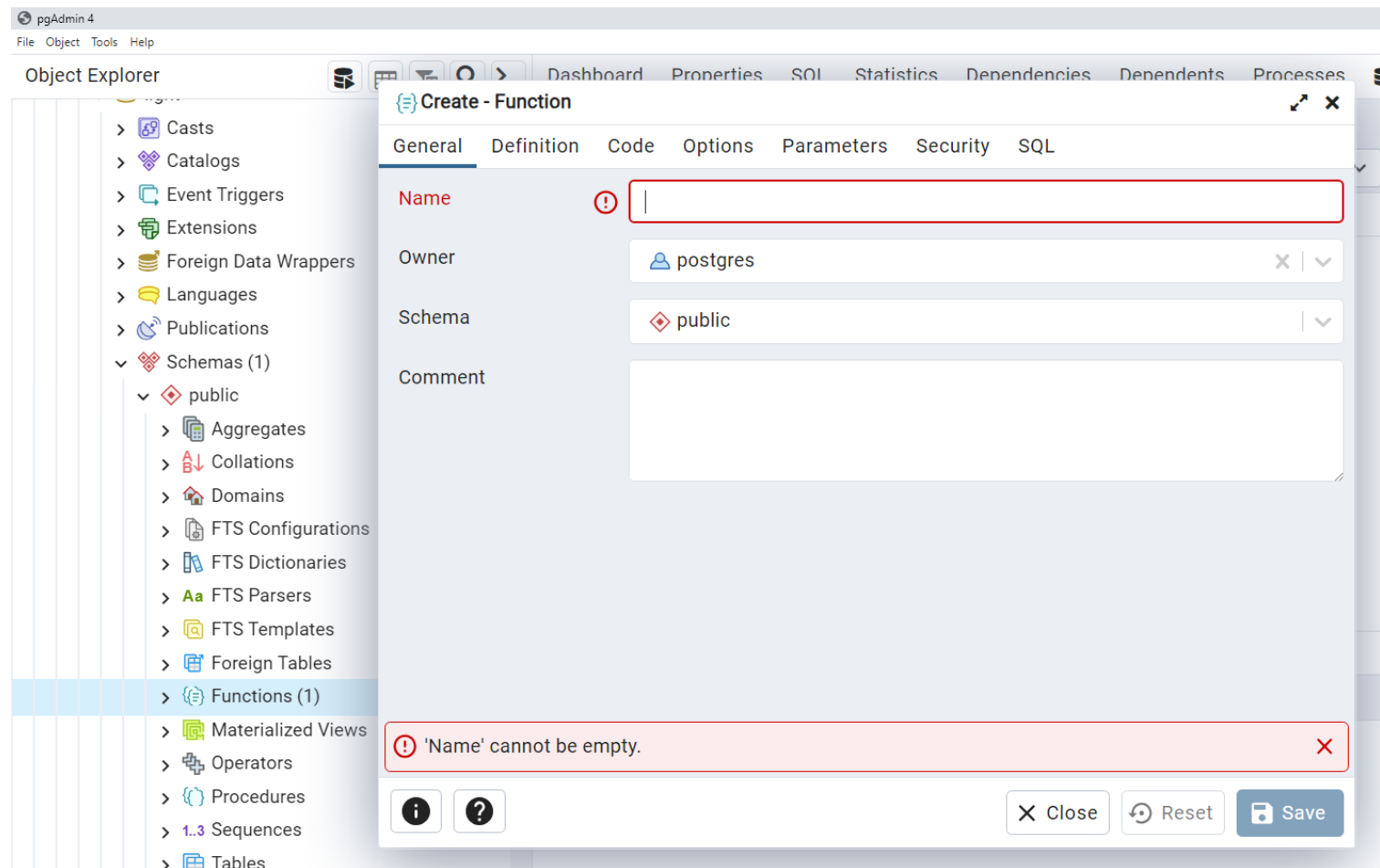
Создание функции в pgAdmin



Нажмите правой кнопкой на Functions в выбранной схеме (в данном случае - public), выберите Create, Function...



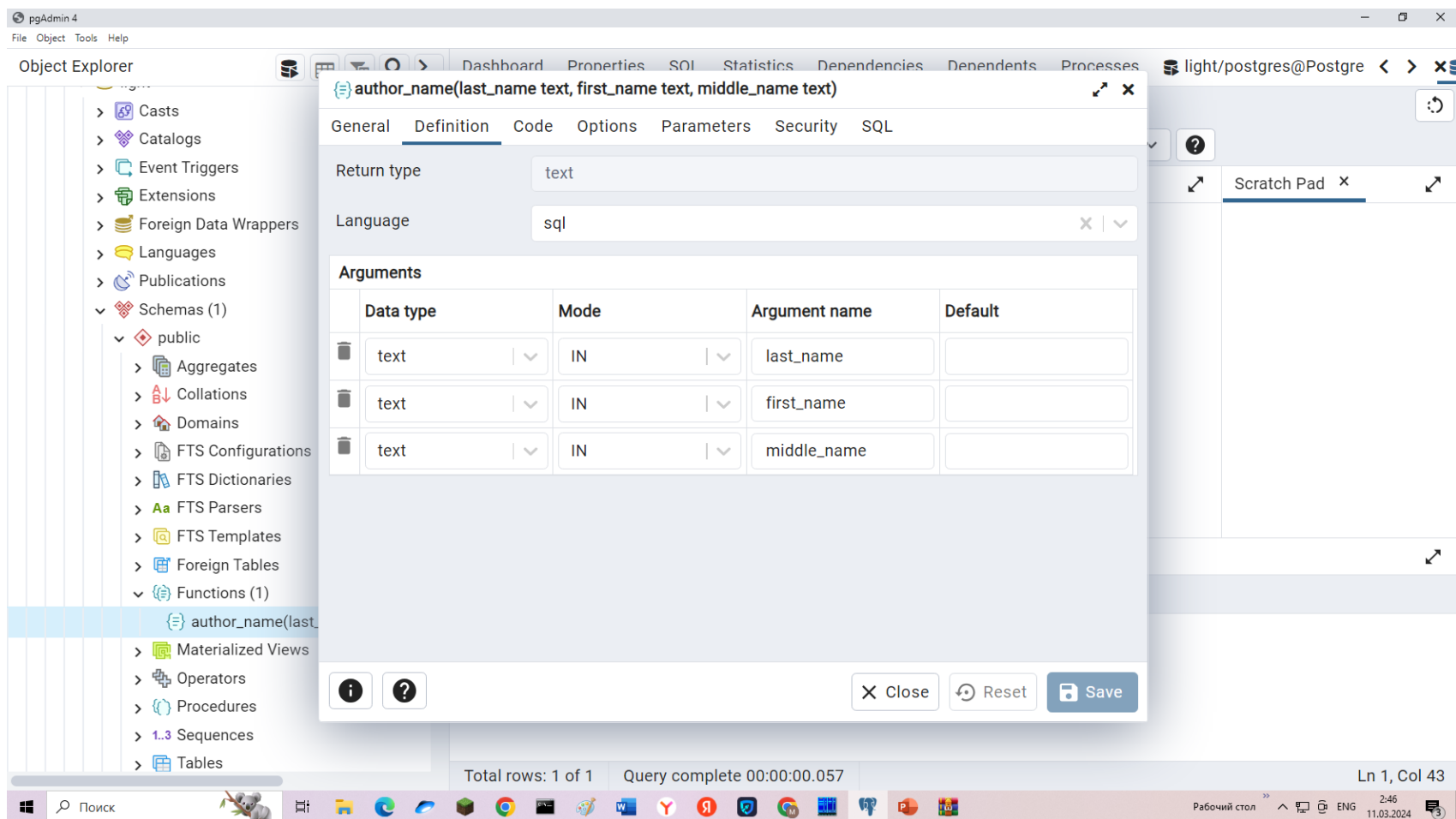
Создание функции в pgAdmin



В появившемся окне на вкладке General введите имя функции.



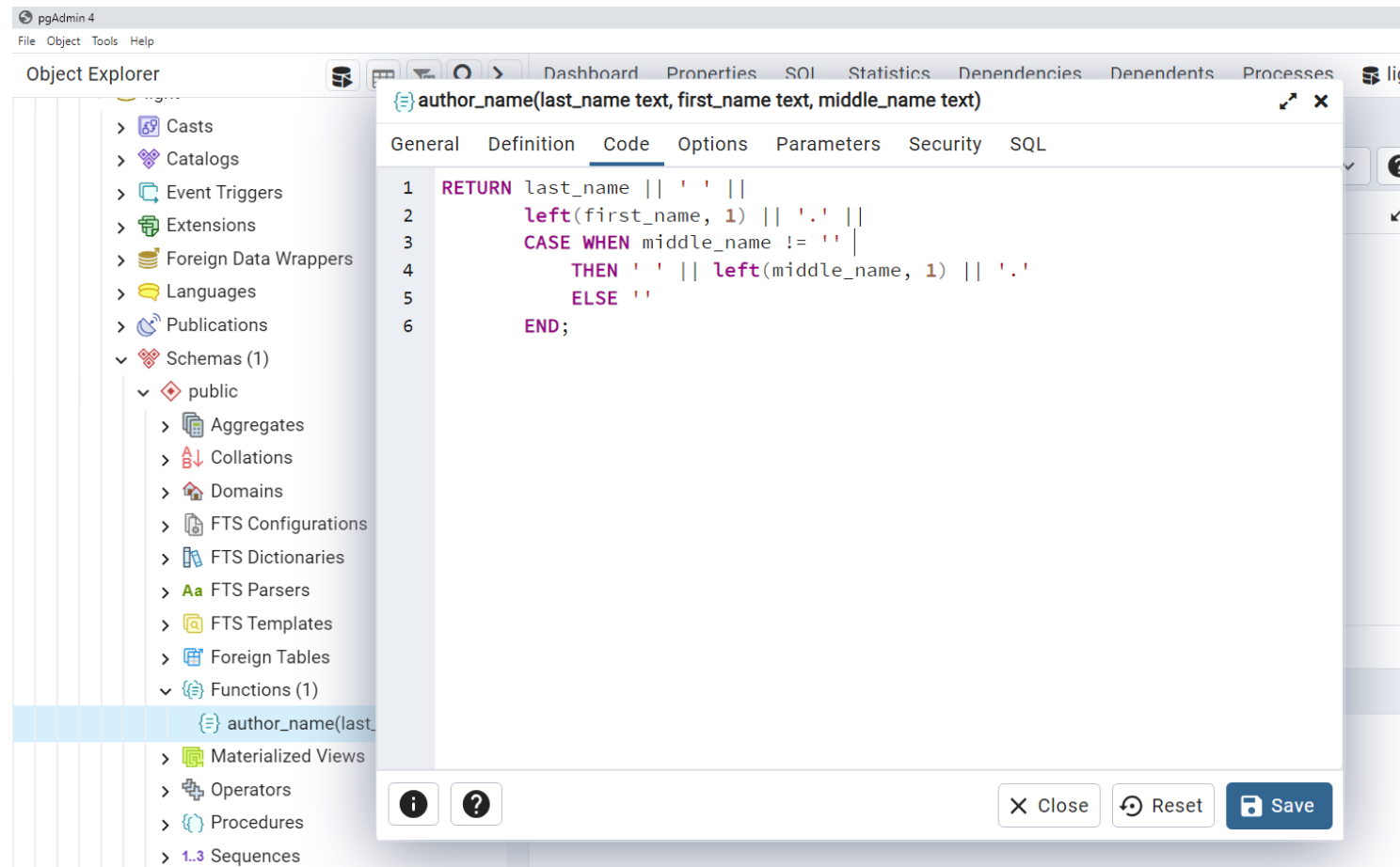
Создание функции в pgAdmin



На вкладке Definition задайте: тип возвращаемого значения (Return type), Язык (Language: сейчас - sql), Параметры (Arguments: тип, имя, режим - необязательно, по умолчанию - если параметр по умолчанию имеет значение).



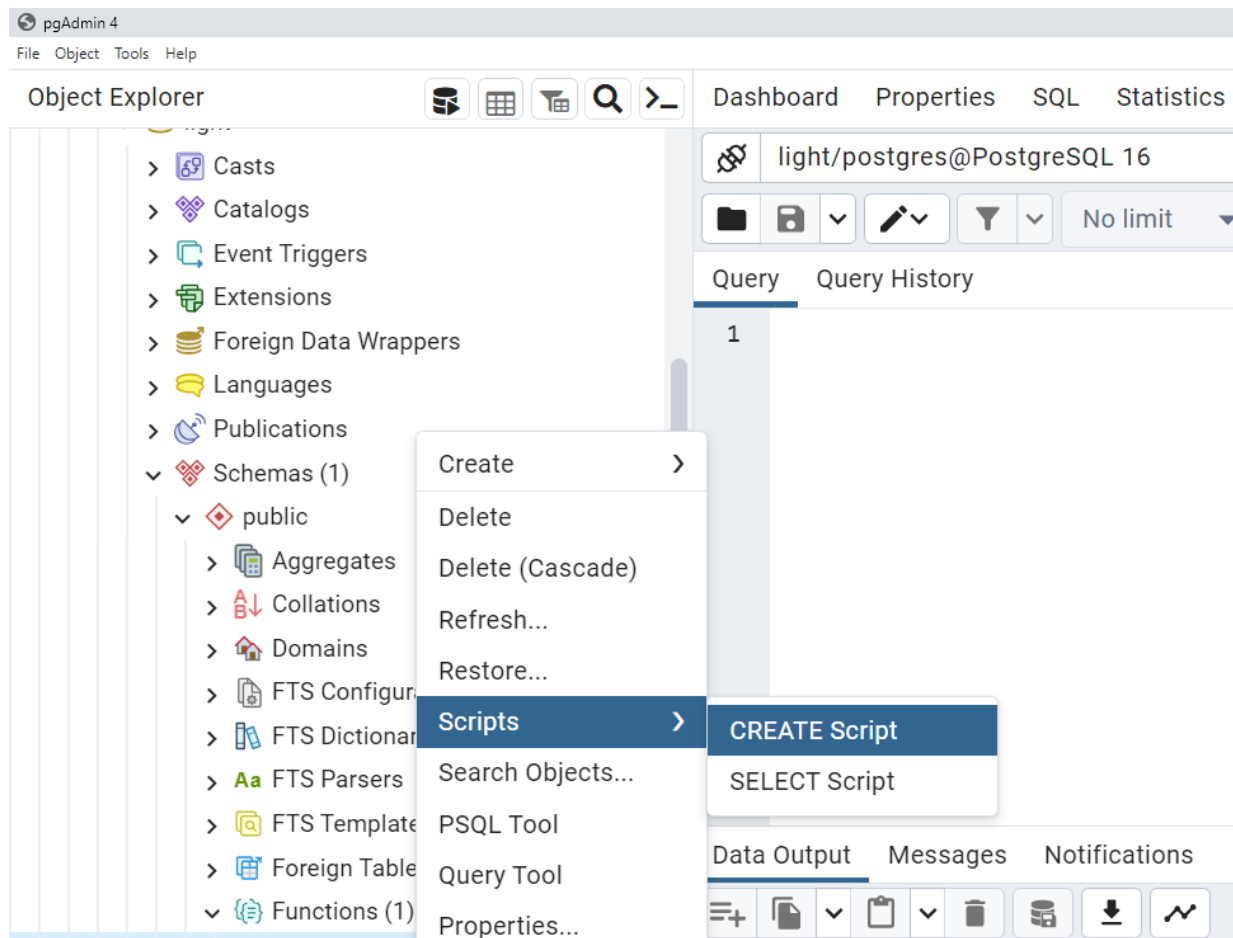
Создание функции в pgAdmin



На вкладке Code
введите код
(тело функции).
Сохраните
изменения.



Создание функции в pgAdmin



Посмотрите созданный скрипт, нажав правой кнопкой на имени функции, появившейся в списке функций (см. скриншот), Scripts, Create Script. Выполните функцию, выбрав/написав соответствующий запрос.



Задания. Вариант 1

Вариант выбирается по формуле: (Номер_по_журналу % 4) + 1, где % - остаток от деления. Все функции/процедуры следует вызывать/выполнять, документировать результаты.

Блок 1

1. Напишите SQL-функцию, в которую передается код самолета, и которая возвращает строку, содержащую через запятые код данного самолета, его название, дату отправления и дату прибытия, номер перелета - для самого длительного перелета.
2. Напишите SQL-функцию, которая для заданного переменного множества flight_id (передается с помощью массива VARIADIC) выдает множество строк, содержащих наименование аэропорта отправления, планируемую дату отправления, статус полета, код самолета.
3. Напишите SQL-функцию, возвращающую количество билетов, проданных на заданный рейс.

Блок 2

1. Создайте копию таблицы bookings. Создайте и выполните SQL-процедуру, которая увеличивает все значения total_amount на процент (от соответствующих значений total_amount), переданный в функцию в качестве параметра.
2. Создайте и выполните SQL-функцию с выходными параметрами, в которую передается номер билета и которая возвращает идентификатор пассажира, имя+фамилию пассажира и контактные данные пассажира в виде трех выходных параметров.
3. Создайте и выполните SQL-функцию с переменным числом аргументов (VARIADIC), которая возвращает сумму переданных аргументов.
4. Создайте и выполните SQL-функцию с параметром fare_cond, порождающую таблицу из тех строк таблицы seats, в которых fare_conditions = fare_cond.
5. Создайте и выполните SQL-функцию с параметром fare_cond, возвращающую множество тех строк таблицы seats, в которых fare_conditions = fare_cond. Реализуйте два варианта: с setoff <имя_таблицы> и с колонками, определяемыми выходными параметрами (setoff record).

Блок 3

1. Создайте функцию на языке PL/pgSQL, в которой переменная с одним и тем же именем объявляется и используется (выводится, ей присваиваются значения) на трех уровнях вложенности, задокументируйте результаты.
2. Придумайте и напишите 3 динамически формируемых команды на языке PL/pgSQL, в которые передаются параметры и используются запросы: select, update, delete к таблицам из БД «Авиарейсы» (предварительно создайте копии таблиц, на которых проводятся эксперименты). В одной из команд задайте имя таблицы динамически.
3. Перепишите на язык PL/pgSQL SQL-функции, реализованные по заданиям 4-5 из блока 2.
4. Напишите процедуру на языке PL/pgSQL, которая просматривает (в цикле) таблицу с информацией о самолетах (aircrafts_data) и выдает все пары моделей самолетов, у которых разница между значениями поля range не превышает заданное значение (передается в качестве параметра).
5. Изучите самостоятельно тему «Курсоры»: <https://postgrespro.ru/docs/postgresql/9.6/plpgsql-cursors#plpgsql-cursor-for-loop>. Создайте курсор с параметром, связанный с запросом, который соединяет flights и airports (эта таблица джойнится дважды) по кодам аэропорта отправления и прибытия. Используя соответствующую курсорную переменную, пройдите в цикле по строкам результата соединения и выведите все строки, в которых город отправления равен заданному городу (заданный город - параметр курсора).

Задания. Вариант 2

Вариант выбирается по формуле: (Номер_по_журналу % 4) + 1, где % - остаток от деления. Все функции/процедуры следует вызывать/выполнять, документировать результаты.

Блок 1

1. Напишите SQL-функцию, которая по номеру билета выдает строку, содержащую passenger_name и contact_data в формате: "Vladimir Frolov, contact_data: phone: ...". Т.е. нужно фамилию и имя из passenger_name из верхнего регистра преобразовать в формат, где первая буква - в верхнем регистре, остальные - в нижнем, а затем сцепить (конкатенировать) результат со словом « contact_data:» и контактными данными.
2. Напишите SQL-функцию, которая для заданного переменного множества ticket_no (передается как массив VARIADIC) выдает множество строк, содержащих id и имя пассажира, amount, fare_conditions.
3. Напишите SQL-функцию, возвращающую количество пассажиров, полетевших эконом-классом на заданном рейсе.

Блок 2

1. Создайте и выполните SQL-функцию с составным типом аргументов, в которую передается переменная составного типа, представляющая строку таблицы bookings, а возвращается уменьшенное на 30% значение total_amount.
2. Создайте и выполните SQL-процедуру, которая добавляет в контактные данные заданных пассажиров заданные номера телефонов (списки идентификаторов пассажиров и их телефонов передаются в качестве параметров). Если не получается передать списки переменной длины, выполните передачу только для изменения фиксированного количества пассажиров - например, 1, 2, 3, - задаете самостоятельно.
3. Создайте и выполните SQL-функцию с переменным числом аргументов (VARIADIC), которая возвращает разность переданных аргументов.
4. Создайте и выполните SQL-функцию с параметром fare_cond, порождающую таблицу из тех строк таблицы ticket_flights, в которых fare_conditions = fare_cond.
5. Создайте и выполните SQL-функцию с параметром fare_cond, возвращающую множество тех строк таблицы ticket_flights, в которых fare_conditions = fare_cond. Реализуйте два варианта: с setoff <имя_таблицы> и с колонками, определяемыми выходными параметрами (setoff record).

Блок 3

1. Создайте функцию на языке PL/pgSQL, в которой переменная с одним и тем же именем объявляется и используется (выводится, ей присваиваются значения) на четырех уровнях вложенности, задокументируйте результаты.
2. Придумайте и напишите 3 динамически формируемых команды на языке PL/pgSQL, в которые передаются параметры и используются запросы: select, update, delete к таблицам из БД «Авиарейсы» (предварительно создайте копии таблиц, на которых проводятся эксперименты). В одной из команд задайте имя таблицы динамически.
3. Перепишите на язык PL/pgSQL SQL-функции, реализованные по заданиям 4-5 из блока 2.
4. Напишите процедуру на языке PL/pgSQL, которая просматривает (в цикле) таблицу с информацией о местах (seats) и выдает все пары строк из нее, в которых номера мест отличаются не более чем двумя последними символами.
5. Изучите самостоятельно тему «Курсоры»: <https://postgrespro.ru/docs/postgresql/9.6/plpgsql-cursors#plpgsql-cursor-for-loop>. Создайте курсор с параметром, связанный с запросом, который соединяет flights и aircrafts. Используя соответствующую курсорную переменную, пройдите в цикле по строкам результата соединения и выведите все строки, в которых модель самолета равна заданной модели (параметру курсора).

Задания. Вариант 3

Вариант выбирается по формуле: (Номер_по_журналу % 4) + 1, где % - остаток от деления. Все функции/процедуры следует вызывать/выполнять, документировать результаты.

Блок 1

1. Напишите SQL-функцию, которая по номеру билета выдает строку, содержащую passenger_name и contact_data в формате: "Vladimir Frolov, contact_data - phone: ...". Т.е. нужно фамилию и имя из passenger_name из верхнего регистра преобразовать в формат, где первая буква - в верхнем регистре, остальные - в нижнем, а затем сцепить (конкатенировать) результат со словом « contact_data - » и контактными данными.
2. Напишите SQL-функцию, которая для заданного переменного множества мест (seat_no) - задается в качестве параметра как массив VARIADIC, выдает множество строк, содержащих код самолета, модель самолета, fare_conditions, seat_no.
3. Напишите SQL-функцию, возвращающую сумму выручки по билетам, проданных на заданный рейс.

Блок 2

1. Создайте копию таблицы ticket_flights. Создайте и выполните SQL-процедуру, которая изменяет все значения amount на значение переданного в функцию параметра.
2. Создайте и выполните SQL-функцию с выходными параметрами, в которую передается номер билета и которая возвращает идентификатор пассажира и контактные данные пассажира в виде двух выходных параметров.
3. Создайте и выполните SQL-функцию с переменным числом аргументов (VARIADIC), которая возвращает результат конкатенации переданных аргументов.
4. Создайте и выполните SQL-функцию с параметром air_code, порождающую таблицу из тех строк таблицы flights, в которых aircraft_code = air_code.
5. Создайте и выполните SQL-функцию с параметром air_code, возвращающую множество тех строк таблицы flights, в которых aircraft_code = air_code. Реализуйте два варианта: с setoff <имя_таблицы> и с колонками, определяемыми выходными параметрами (setoff record).

Блок 3

1. Создайте функцию на языке PL/pgSQL, в которой переменная с одним и тем же именем объявляется и используется (выводится, ей присваиваются значения) на пяти уровнях вложенности, задокументируйте результаты.
2. Придумайте и напишите 3 динамически формируемых команды на языке PL/pgSQL, в которые передаются параметры и используются запросы: select, update, delete к таблицам из БД «Авиарейсы» (предварительно создайте копии таблиц, на которых проводятся эксперименты). В одной из команд задайте имя таблицы динамически.
3. Перепишите на язык PL/pgSQL SQL-функции, реализованные по заданиям 4-5 из блока 2.
4. Напишите и выполните процедуру на языке PL/pgSQL, которая просматривает (в цикле) таблицу с информацией о билетах и полетах (tickets_flights) и выдает все пары строк из нее, в которых разница между значениями amount больше заданного значения (заданное значение - параметр процедуры).
5. Изучите самостоятельно тему «Курсоры»: <https://postgrespro.ru/docs/postgresql/9.6/plpgsql-cursors#plpgsql-cursor-for-loop>. Создайте курсор с параметром, связанный с запросом, который соединяет flights и ticket_flights. Используя соответствующую курсорную переменную, пройдите в цикле по строкам результата соединения и выведите все строки, в которых дата отправления равна заданной дате (параметру курсора).

Задания. Вариант 4

Вариант выбирается по формуле: (Номер_по_журналу % 4) + 1, где % - остаток от деления. Все функции/процедуры следует вызывать/выполнять, документировать результаты.

Блок 1

1. Напишите SQL-функцию, которая по номеру билета выдает строку, содержащую passenger_id, passenger_name и contact_data в формате: "23423424 - Vladimir Frolov, contact_data: phone: ...". Т.е. нужно фамилию и имя из passenger_name из верхнего регистра преобразовать в формат, где первая буква - в верхнем регистре, остальные - в нижнем, результат должен представлять собой конкатенацию id пассажира, тире (и пробелов вокруг него), измененного passenger_name и т.д.
2. Напишите SQL-функцию, которая для переменного числа моделей самолетов (передаются как массив VARIADIC) выдает количества пассажиров, купивших билеты на места класса Economy.
3. Напишите SQL-функцию, возвращающую суммы выручек по дням по билетам, проданным на заданный рейс.

Блок 2

1. Создайте копию таблицы ticket_flights. Создайте и выполните SQL-процедуру, которая изменяет все значения amount, меньшие среднего [значения amount], уменьшает на заданный процент (один параметр), а все значения amount, большие среднего, увеличивает на заданный процент (другой параметр).
2. Создайте и выполните SQL-функцию с выходными параметрами, в которую передается номер билета и которая возвращает идентификатор пассажира и контактные данные пассажира в виде двух выходных параметров.
3. Создайте и выполните SQL-функцию с переменным числом аргументов (VARIADIC), которая возвращает результат конкатенации переданных аргументов.
4. Создайте и выполните SQL-функцию с параметром airport_num, порождающую таблицу из тех строк таблицы flights, в которых код аэропорта прибытия равен airport_num.
5. Создайте и выполните SQL-функцию с параметром air_code, возвращающую множество тех строк таблицы seats, в которых aircraft_code = air_code. Реализуйте два варианта: с setoff <имя_таблицы> и с колонками, определяемыми выходными параметрами (setoff record).

Блок 3

1. Создайте функцию на языке PL/pgSQL, в которой переменная с одним и тем же именем объявляется и используется (выводится, ей присваиваются значения) на четырех уровнях вложенности, задокументируйте результаты.
2. Придумайте и напишите 3 динамически формируемых команды на языке PL/pgSQL, в которые передаются параметры и используются запросы: select, update, delete к таблицам из БД «Авиарейсы» (предварительно создайте копии таблиц, на которых проводятся эксперименты). В одной из команд задайте имя таблицы динамически.
3. Перепишите на язык PL/pgSQL SQL-функции, реализованные по заданиям 4-5 из блока 2.
4. Напишите и выполните процедуру на языке PL/pgSQL, которая просматривает (в цикле) таблицу с информацией о бронированиях(bookings) и выдает все пары строк из нее, в которых разница между значениями total_amount меньше заданного значения (заданное значение - параметр процедуры).
5. Изучите самостоятельно тему «Курсоры»: <https://postgrespro.ru/docs/postgresql/9.6/plpgsql-cursors#plpgsql-cursor-for-loop>. Создайте курсор с параметром, связанный с запросом, который соединяет flights и ticket_flights. Используя соответствующую курсорную переменную, пройдите в цикле по строкам результата соединения и выведите все строки, в которых дата прибытия равна заданной дате (заданная дата - параметр курсора).